

IDOR VULNERABILITY ASSESSMENT OF A VULNERABLE BANKING WEB APPLICATION

SECURITY ASSESSMENT REPORT

Organization:

Vulnbank - The modern banking Platform

Prepared by:

Peter oriasotie

Environment:

Authorized Vulnerable Training Laboratory

Assessment Type:

Web Application Security Assessment

Date:

26 June 2026

**CONFIDENTIAL — TRAINING/EDUCATIONAL USE

Table of Contents

Executive Summary.....	1
Introduction	2
Scope.....	3
Objectives.....	4
Methodology.....	5
Vulnerability Identification.....	6
Proof of Concept.....	7
Impact Assessment.....	8
Severity Assessment.....	9
Root Cause.....	10
Remediation.....	11
Conclusion.....	12
References.....	13
Appendix.....	14

1. Executive Summary

An authorized security assessment was conducted against a deliberately vulnerable banking web application within a controlled training laboratory environment. The assessment focused on identifying weaknesses in the application's access-control mechanisms, particularly those affecting sensitive transaction information.

During testing, an **Insecure Direct Object Reference (IDOR)**, also referred to as **Broken Object-Level Authorization (BOLA)**, vulnerability was identified within the transaction-history functionality. The application was found to accept a client-controlled account identifier without adequately verifying whether the authenticated user was authorized to access the corresponding account.

The vulnerability was successfully validated using two separate test accounts. While authenticated as one test user, the account identifier in the transaction-history request was modified to reference a second test user's account. The application subsequently returned transaction-history information belonging to the second user, demonstrating unauthorized access to another user's data.

The primary impact of this vulnerability is the **unauthorized disclosure of sensitive financial transaction information**. In a real-world banking environment, exploitation of this weakness could result in significant privacy concerns, exposure of confidential financial activity, loss of customer trust, and potential regulatory or compliance implications.

It is recommended that the application implement robust **server-side object-level authorization checks** for all account and transaction-related requests. The server should verify that the authenticated user is authorized to access the requested account before returning any associated information. Client-supplied account identifiers should not be treated as sufficient evidence of authorization.

Testing was performed exclusively against the authorized vulnerable training environment and the designated test accounts.

2. Introduction

Background:

Web applications that handle sensitive information must implement effective authentication and authorization mechanisms to ensure that users can access only the resources they are permitted to access. Authentication determines the identity of a user, while authorization determines what that authenticated user is allowed to access.

Weaknesses in authorization controls can allow authenticated users to access resources belonging to other users. In applications that process financial information, such weaknesses can expose sensitive transaction and account information.

vulnerability overview:

Insecure Direct Object Reference (IDOR) is an access-control vulnerability that occurs when an application uses a user-controlled identifier to directly reference an internal object, such as an account, transaction, document, or user record, without properly verifying that the requesting user is authorized to access that object.

The vulnerability is commonly associated with **Broken Object-Level Authorization (BOLA)**. An application may correctly authenticate a user but fail to verify whether that user has permission to access the specific object requested.

For example, if User A is authenticated and requests their own transaction information, the application should verify that User A is authorized to access the requested account. If User A can modify an account identifier and retrieve User B's transaction information, the application is not properly enforcing object-level authorization.

Assessment Context:

This assessment was performed against a deliberately vulnerable banking web application within an authorized training laboratory environment.

Two separate test accounts were used during the assessment. One account represented the authenticated testing user, while the second account was used to determine whether access controls prevented unauthorized access to another user's transaction information.

All testing was restricted to the designated training environment.

Purpose of this Assessment:

The purpose of this assessment was to evaluate the application's object-level authorization controls within the transaction-history functionality.

The assessment specifically aimed to determine whether an authenticated user could manipulate a client-controlled account identifier to access transaction information belonging to another user.

The assessment also aimed to document the vulnerability, demonstrate its security impact, identify the likely root cause, and provide appropriate remediation recommendations.

3. Scope

3.1 In-Scope:

The assessment was limited to the designated vulnerable banking web application provided within the authorized training laboratory environment.

The following components were included within the scope of testing:

Scope Item	Description
Target Domain	https://vulnbank.org
Organization	Vulnbank
Functionality Tested	Transaction-history functionality
Access Control	Object-level authorization
User Accounts	Two designated test accounts
Testing Type	Authenticated web application security testing
Primary Vulnerability	IDOR / Broken Object-Level Authorization

The assessment focused specifically on determining whether one authenticated test account could access transaction information associated with another test account by manipulating a client-controlled object/account identifier.

3.2 Out-of-Scope:

The following activities were outside the scope of this assessment:

- Testing real banking systems or production environments
- Accessing accounts belonging to real users
- Denial-of-service or availability testing
- Social engineering or phishing
- Physical security testing
- Attacks against third-party systems
- Access to data outside the designated training environment
- Destructive actions that could affect the availability or integrity of the laboratory application

3.3 Testing Boundaries:

Testing was conducted exclusively within the authorized training laboratory environment.

Two separate test accounts were used to validate the access-control behavior. The assessment did not involve real customer accounts or production banking infrastructure.

Testing was limited to demonstrating whether the transaction-history endpoint properly enforced authorization between the two designated test accounts. No attempt was made to access or modify information outside the defined assessment scope.

4. Assessment Objectives

The primary objective of this assessment was to evaluate the security of the vulnerable banking web application, with particular focus on access-control mechanisms protecting user account and transaction information.

The assessment objectives were to:

1. **Identify access-control weaknesses** within the application's transaction-history functionality.
2. **Determine whether object-level authorization was properly enforced** when authenticated users requested account or transaction information.
3. **Identify client-controlled object identifiers** used by the application's transaction-related endpoints.
4. **Test whether object identifiers could be manipulated** to determine whether an authenticated user could access resources belonging to another test account.
5. **Validate and document any identified vulnerability** using controlled requests and responses within the authorized training environment.
6. **Assess the potential security impact** of unauthorized access to another user's transaction information.
7. **Determine the likely root cause** of the identified access-control weakness.
8. **Provide appropriate remediation recommendations** to strengthen server-side authorization controls.
9. **Define a retesting procedure** to verify that the vulnerability has been properly addressed

fol# 5. Assessment Methodology

5 Methodology Overview

The assessment followed a structured web application security testing process designed to identify and validate access-control weaknesses within the authorized vulnerable banking application.

The methodology focused on the transaction-history functionality and examined whether the application correctly enforced authorization when users requested transaction information associated with different accounts.

The assessment process consisted of the following phases:

Application Reconnaissance

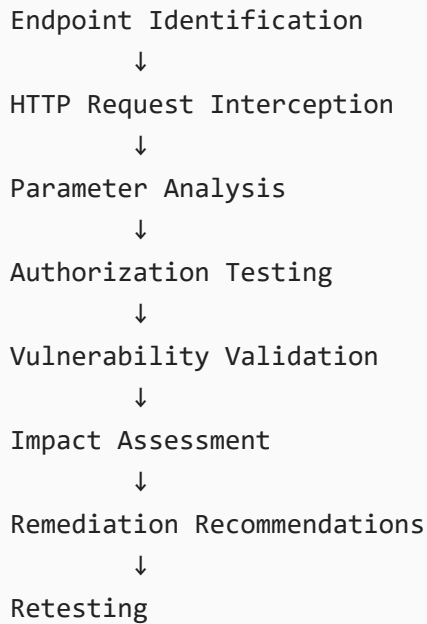


Authentication



Application Mapping





5.2 Application Reconnaissance

The assessment began with an examination of the application's available functionality after authentication.

The objective of this phase was to understand the application's structure and identify functionality that handled sensitive user information.

Particular attention was given to account-related and transaction-related functionality because these areas may contain sensitive financial information and require strong authorization controls.

5.3 Authentication and Test Accounts

Two separate test accounts were used during the assessment.

One account represented the authenticated testing user, while the second account was used to verify whether access controls prevented unauthorized access to another user's resources.

The accounts were maintained separately on different device throughout testing to ensure that the authorization behavior could be accurately evaluated.

5.4 Application Mapping

The application's transaction-related functionality was examined to identify the HTTP requests generated when viewing transaction information.

The purpose of this phase was to determine:

- Which endpoints were responsible for retrieving transaction information.
- Which HTTP methods were used.
- Which parameters were submitted by the client.
- Whether account or object identifiers were present within requests.
- How the application responded to different requests.

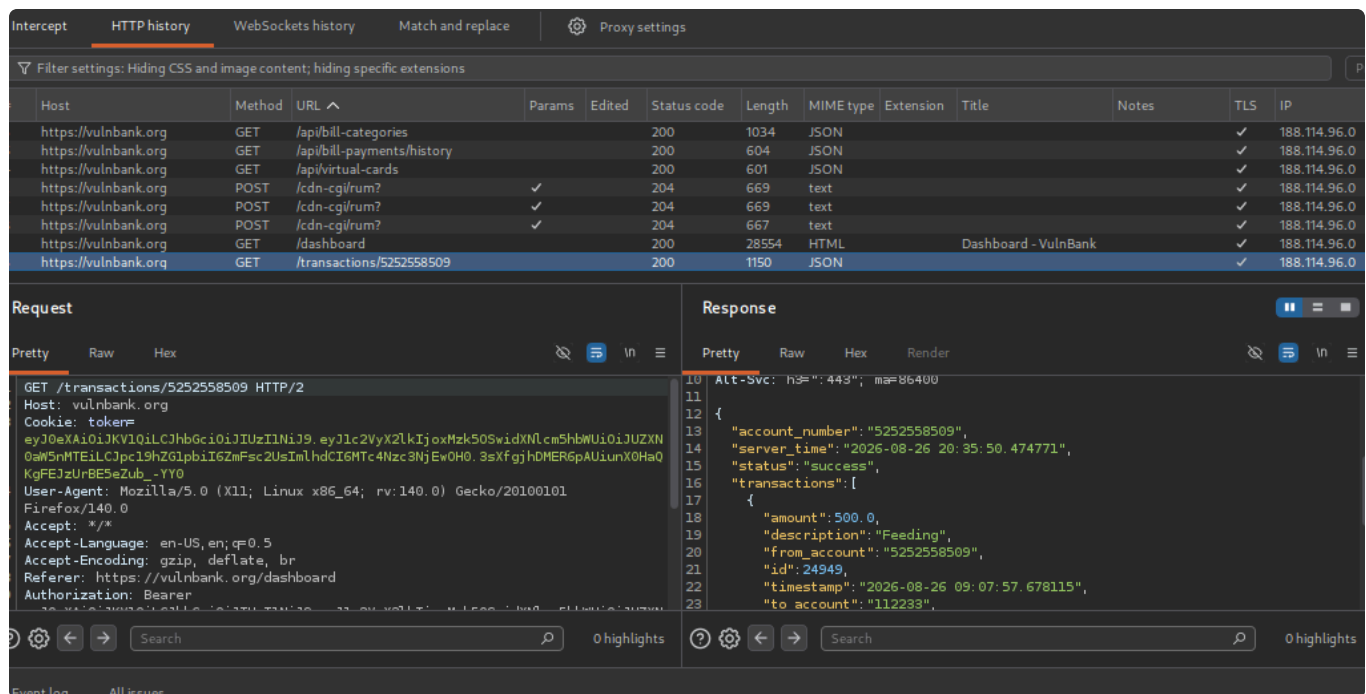
5.5 HTTP Request Interception

Burp Suite was used to intercept and inspect HTTP requests generated by the application.

The transaction-history request was captured and analyzed to identify parameters that influenced which account or transaction object was returned by the server.

A baseline request was first recorded while authenticated as the designated testing user.

Figure 1 — Transaction-history request captured in Burp Suite



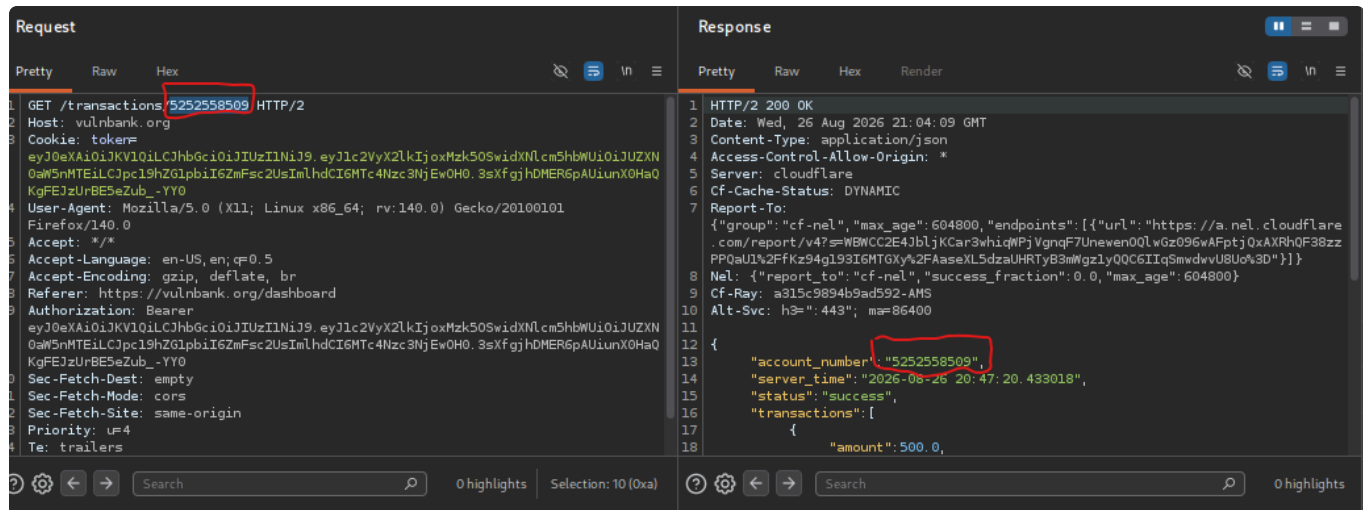
5.6 Parameter Analysis

The intercepted request was examined for client-controlled parameters associated with the requested account.

An account/object identifier was identified within the transaction-history request.

The identifier appeared to determine which account's transaction information was retrieved by the application.

Figure 2 — Account/object identifier identified within the request



6. Vulnerability Identification

6.1 Initial Observation

During analysis of the transaction-history request, an account/object identifier was observed within the request parameters.

The identifier corresponded to the account associated with the currently authenticated test user.

Because the identifier was supplied by the client, it was examined to determine whether the application performed appropriate authorization checks before returning the requested transaction information.

6.2 Object Identifier Manipulation

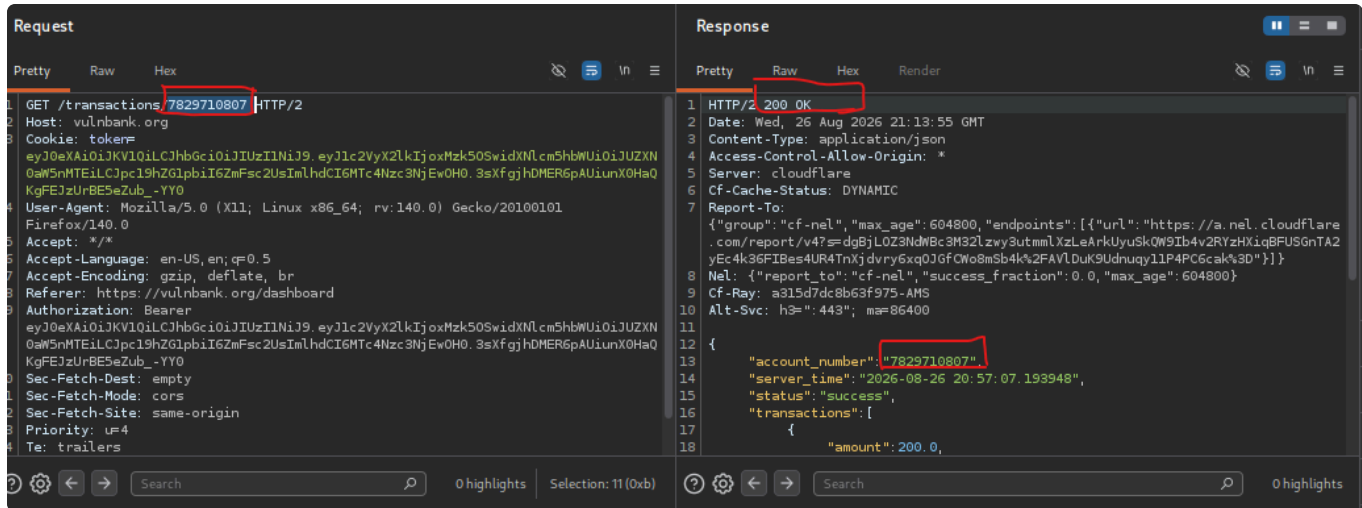
After identifying the account/object identifier, controlled authorization testing was performed.

The identifier associated with the authenticated test account was replaced with the identifier belonging to the second designated test account.

The modified request was then submitted to the application while maintaining the original authenticated session.

The purpose of this test was to determine whether the server verified that the authenticated user was authorized to access the requested account.

Figure 3 — Modified account/object identifier



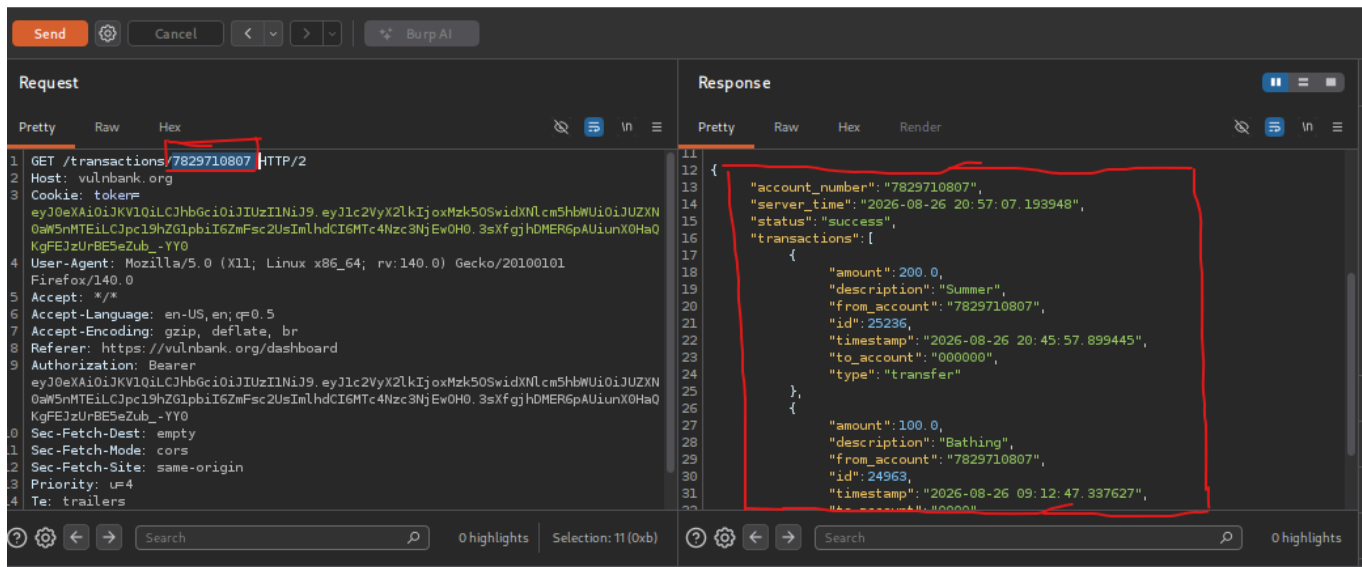
6.3 Unauthorized Access

Following submission of the modified request, the application returned transaction-history information associated with the second test account.

The authenticated session remained associated with the first test account.

This demonstrated that the application allowed an authenticated user to access another user's transaction information by manipulating a client-controlled object identifier.

Figure 4 — Unauthorized transaction information returned



6.4 Vulnerability Classification

The observed behavior is classified as an **Insecure Direct Object Reference (IDOR) / Broken Object-Level Authorization (BOLA)** vulnerability.

The issue occurs because the application does not adequately verify whether the authenticated user is authorized to access the account represented by the supplied identifier.

7. Proof of Concept

7.1 Legitimate Request

The following request represents the normal transaction-history request made while authenticated as User A.

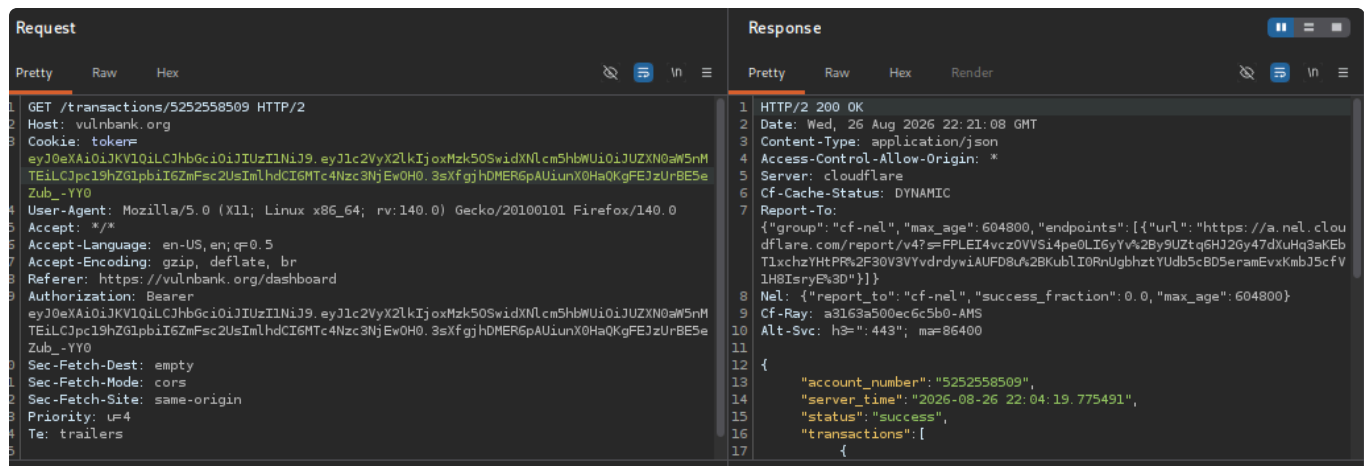
GET /transactions/5252558509 HTTP/2

Host: Vulnbank.org

cookies:

eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJlc2VyX2lkIjoxMzk5OStwidXNlcm5hbWUiOiJUZWN0aW5nMTEiLCJpc19hZG1pbil6ZmFsc2UsImhhdCIGMTc4Nzc3NjEwOHO.3sXfgjhDMER6pAUiunXOHaQKgFEJzUrBESeZub_-YYO

Expected behavior: The application returns User A's transaction information.



7.2 Modified Request

The account identifier was modified to reference User B's test account while maintaining User A's authenticated session.

GET /transactions/7829710807 HTTP/2

Host: Vulnbank.org

cookies:

eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJlc2VyX2lkIjoxMzk5OStwidXNlcm5hbWUiOiJUZWN0aW5nMTEiLCJpc19hZG1pbil6ZmFsc2UsImhhdCIGMTc4Nzc3NjEwOHO.3sXfgjhDMER6pAUiunXOHaQKgFEJzUrBESeZub_-YYO

The screenshot displays a network tool interface with two main panels: 'Request' and 'Response'.

Request Panel:

- Method:** GET
- URL:** /transactions/7829710807
- Host:** vulnbank.org
- Cookie:** token=eyJ0eXAiOiJKV1QiOiJCLlhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoxMzk5S05widXNlcm5hbWUiOiJ1UzN0aW5nMTEiLCJpc19hZG1pb1I6ZmFsc2UsImhhdCI6MTc4Nzc3IjEwOHO.3sXfgjhdMER6pAUiuXOHaQKGFzUrBE5eZub_-YY0
- User-Agent:** Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
- Accept:** */*
- Accept-Language:** en-US,en;q=0.5
- Accept-Encoding:** gzip, deflate, br
- Referer:** https://vulnbank.org/dashboard
- Authorization:** Bearer eyJ0eXAiOiJKV1QiOiJCLlhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoxMzk5S05widXNlcm5hbWUiOiJ1UzN0aW5nMTEiLCJpc19hZG1pb1I6ZmFsc2UsImhhdCI6MTc4Nzc3IjEwOHO.3sXfgjhdMER6pAUiuXOHaQKGFzUrBE5eZub_-YY0
- Sec-Fetch-Dest:** empty
- Sec-Fetch-Mode:** cors
- Sec-Fetch-Site:** same-origin
- Priority:** u=4
- Te:** trailers

Response Panel:

- Status:** 200 OK
- Date:** Wed, 26 Aug 2026 22:28:35 GMT
- Content-Type:** application/json
- Access-Control-Allow-Origin:** *
- Server:** cloudflare
- CF-Cache-Status:** DYNAMIC
- Report-To:**

```
{
  "group": "cf-nel",
  "max_age": 604800,
  "endpoints": [
    {
      "url": "https://a.nel.cloudflare.com/report/v4?sm=8vBKFR06JRGAvIPZua1AQz2BPy09xLF5WuXrYz2BnsOf49KXVmw3%2Fwrt%2F0pM7JHv1le%2BKBJUusQW503egGiEnoaAV08yb2c%2FG16J3s3RMScfMwWemvXKTQ8zw6BFdU3D"
    }
  ]
}
```
- Nel:** {"report_to": "cf-nel", "success_fraction": 0.0, "max_age": 604800}
- CF-Ray:** a316453b6fc96db1-AMS
- Alt-Svc:** h3=":443"; ma=86400
- Body:**

```
{
  "account_number": "7829710807",
  "server_time": "2026-08-26 22:11:47.039650",
  "status": "success",
  "transactions": [
    {

```

This response demonstrated that the server processed the modified object identifier without adequately verifying whether User A was authorized to access User B's account.

The screenshot shows a successful HTTP transaction in Burp Suite. The Request tab displays a GET request to /transactions/7829710807 on vulnbank.org. The Response tab shows a 200 OK status with JSON data containing two transactions. Red annotations highlight the transaction details.

Request:

```
GET /transactions/7829710807 HTTP/2
Host: vulnbank.org
Cookie: token=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsInR5cCI6IHYXZlkIjxvMzk1S0w5dXNlcmShbWUiOiJUZXRhbnQwSWNmTElJCjpc19hZGlpbSI6ZmFsc2UsImhhdCIGMTc4Nzc3NjEwOHO.3sXfgjhDMER6pAUiuNXOHAKgFEJzUrBE5eZub_-YY0
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://vulnbank.org/dashboard
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsInR5cCI6IHYXZlkIjxvMzk1S0w5dXNlcmShbWUiOiJUZXRhbnQwSWNmTElJCjpc19hZGlpbSI6ZmFsc2UsImhhdCIGMTc4Nzc3NjEwOHO.3sXfgjhDMER6pAUiuNXOHAKgFEJzUrBE5eZub_-YY0
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
Priority: u=4
Te: trailers
```

Response:

```
HTTP/2 200 OK
Content-Type: application/json
{
  "account_number": "7829710807",
  "server_time": "2026-08-26 22:14:59.790285",
  "status": "success",
  "transactions": [
    {
      "amount": 200.0,
      "description": "Summer",
      "from_account": "7829710807",
      "id": 25236,
      "timestamp": "2026-08-26 20:45:57.899445",
      "to_account": "000000",
      "type": "transfer"
    },
    {
      "amount": 100.0,
      "description": "Bathing",
      "from_account": "7829710807",
      "id": 24963,
      "timestamp": "2026-08-26 09:12:47.337627",
      "to_account": "00000",
      "type": "transfer"
    }
  ]
}
```

User A authenticated → User A requests transaction data → Account identifier modified to User B → Request submitted using User A's session → User B's transaction information returned.

The behavior confirms an object-level authorization weakness within the affected transaction-history functionality.

8. Impact Assessment

8.1 Confidentiality Impact

The primary demonstrated impact is unauthorized disclosure of financial transaction information.

An authenticated user was able to access transaction-history information belonging to another test account without appropriate authorization.

8.2 Privacy Impact

Transaction histories may contain sensitive information about an individual's financial activity. Unauthorized disclosure could therefore represent a significant privacy concern.

**8.3 Potential Account Enumeration

If account identifiers are predictable or can otherwise be obtained, an attacker could potentially attempt to access additional account objects.

The extent of this potential impact would depend on the application's identifier structure and the authorization controls applied to other endpoints.

**8.4 Business Impact

In a real-world banking environment, unauthorized access to financial information could result in:

- Customer privacy violations.
- Loss of customer trust.
- Reputational damage.
- Regulatory or compliance concerns.
- Increased risk of further attacks against exposed functionality.

The assessment demonstrated unauthorized **read access** to transaction information. Modification or deletion of another user's information was not demonstrated during this assessment.

9 Severity / Risk Rating

9.1 Suggested Severity

Severity: High

The vulnerability is considered high risk within the context of this training scenario because an authenticated user can access sensitive financial transaction information belonging to another user.

9.2 Risk Factors

Factor	Assessment
Authentication Required	Yes
Authorization Bypass	Yes
Data Exposure	Financial transaction information
Confidentiality Impact	High
Integrity Impact	Not demonstrated
Availability Impact	Not demonstrated
User Interaction	Not required for demonstrated access
Suggested Severity	High

**9.3 Severity Justification

The primary security consequence is unauthorized access to sensitive information belonging to another user.

The vulnerability does not require bypassing authentication because the attacker can operate using a valid authenticated account. Instead, the weakness occurs because the application fails to properly enforce authorization at the object level.

The final severity should be adjusted if additional testing demonstrates a different scope or impact.

10. Root Cause Analysis

The likely root cause of the vulnerability is insufficient server-side object-level authorization.

The application accepts an account/object identifier supplied by the client and uses it to determine which transaction information should be returned.

However, the application does not adequately verify that the authenticated user is authorized to access the account represented by that identifier.

The authorization decision should be based on the authenticated user's permissions and the ownership or access rights associated with the requested object.

The application should not treat possession of a valid account identifier as sufficient authorization to access the associated information.

11. Remediation

**11.1 Server-Side Authorization

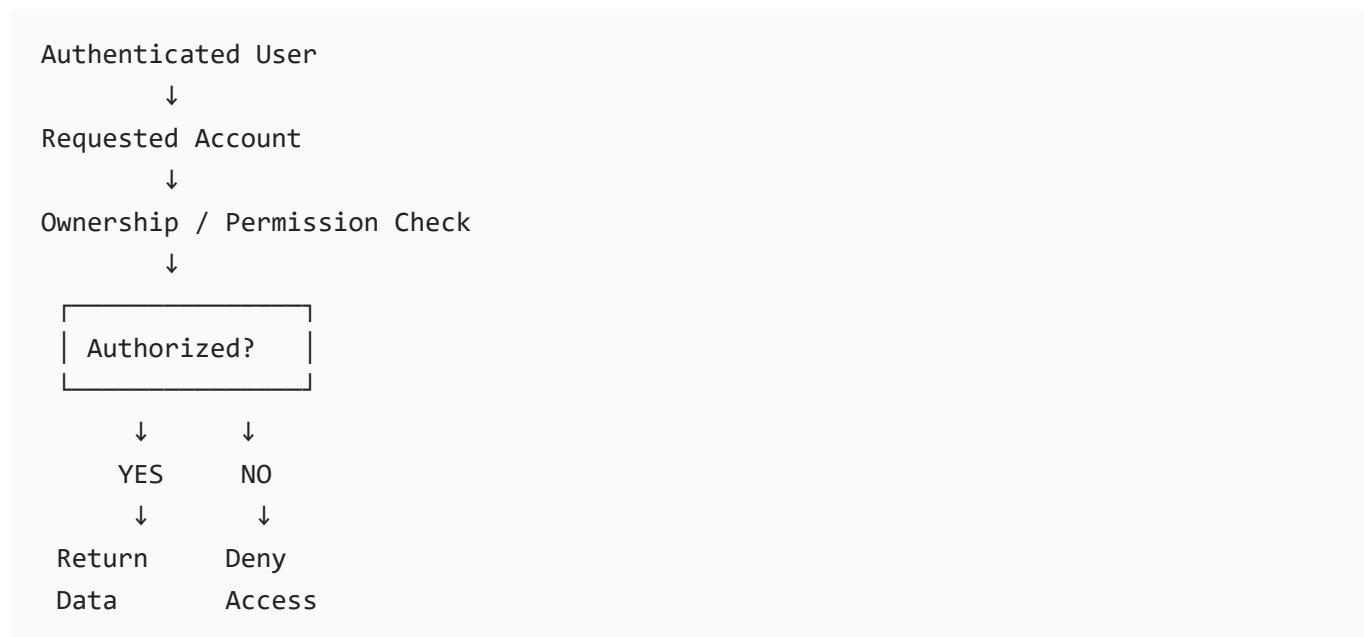
Implement server-side authorization checks for every request involving account and transaction objects.

The server should identify the authenticated user and verify that the user has permission to access the requested account before returning any information.

**11.2 Object Ownership Validation

The application should verify the relationship between the authenticated user and the requested account.

For example:



**11.3 Do Not Trust Client-Controlled Identifiers

Account identifiers supplied by the client should be treated as references, not as proof of authorization.

Authorization must be determined by server-side controls.

**11.4 Consistent Authorization Controls

The same authorization principle should be applied consistently across all sensitive endpoints, including:

- Transaction history.
- Account information.
- Account statements.
- Payment information.
- User profiles.
- Other sensitive financial objects.

**11.5 Error Handling

Unauthorized requests should be rejected and should not disclose sensitive information.

An appropriate response may be:

```
HTTP/1.1 403 Forbidden
```

The application should also avoid returning sensitive information in error messages.

12. Conclusion

The security assessment identified an **Insecure Direct Object Reference (IDOR) / Broken Object-Level Authorization (BOLA)** vulnerability within the transaction-history functionality of the vulnerable banking web application.

The vulnerability was successfully demonstrated using two separate authorized test accounts. While authenticated as one test user, the account/object identifier within the transaction request was modified to reference another test account. The application subsequently returned the second user's transaction-history information.

The finding demonstrates a failure to adequately enforce object-level authorization on the affected functionality.

The primary impact is unauthorized disclosure of sensitive financial transaction information. The recommended remediation is to implement robust server-side authorization and ownership validation for every request involving sensitive account and transaction objects.

Testing was conducted exclusively within the authorized training laboratory environment.

13. References

1. OWASP Foundation — OWASP Top 10: Broken Access Control.
2. OWASP Foundation — Web Security Testing Guide.
3. MITRE — CWE-639: Authorization Bypass Through User-Controlled Key.
4. FIRST — Common Vulnerability Scoring System (CVSS).

14. Appendix

**Appendix A — Testing Evidence

The following evidence was collected during the assessment:

- Legitimate transaction-history request.
- Identified account/object parameter.
- Modified account/object parameter.
- Unauthorized transaction-history response.
- Additional Burp Suite request/response evidence.

**Appendix B — Sanitization

All screenshots and HTTP requests included in the publicly distributed version of this report should be reviewed and sanitized before publication.

The following information should be removed or redacted where applicable:

- Passwords.
- Session cookies.
- Authorization tokens.
- API keys.
- CSRF tokens.
- Personal information.
- Unnecessary account information.
- Any information unrelated to the security finding.
- **Appendix C — Key Finding Summary

Finding	Description	Severity	Status
IDOR / BOLA	Authenticated User A can access User B's transaction information by manipulating an account/object identifier.	High	Identified / Validated

End of Report

Prepared by: PETER ORIASOTIE

Assessment Type: Authorized Training Laboratory Security Assessment

Report Version: 1.0